AUTOBOUGHT · INGENIERÍA DE SOFTWARE II

Dashboard del vendedor con Streamlit

Qué se hizo y por qué se decidió así · HU 5.1 Acceder al dashboard · HU 5.2 Visualizar stock actual · HU 5.3 Visualizar productos más vendidos

Rama `feat/dashboard-streamlit` · 24/10/2026

1. Resumen

El dashboard interno para vendedores quedó hecho con Streamlit, como quería el equipo. Es una app aparte, en la carpeta `analitica/`, que **no lee la base de datos**: le pide los datos a dos endpoints nuevos del backend (`/api/v1/analytics/stock` y `/api/v1/analytics/best-sellers`), que solo responden a cuentas de vendedor. Desde la tienda, el vendedor llega con el botón «Abrir dashboard» del portal `/dashboard`.

El dashboard tiene una fila de indicadores (repuestos, unidades, con stock bajo, sin stock) y dos pestañas: **Stock actual**, con todos los repuestos, búsqueda y filtros, y **Más vendidos**, con un ranking por unidades vendidas en el período elegido.

> **Estado:** implementado y probado. Pasan todos los tests: 384 del backend, 16 del dashboard y 340 del frontend. También se probó con un backend real cargado con los tres seeds. Los cambios están en la rama local, todavía sin commitear.

2. Punto de partida: qué había antes

Antes de escribir código revisé el repo y todas las ramas remotas. De Streamlit no había nada en ninguna; lo que sí estaba sirvió de base:

Pieza existente	Qué tenía	Qué hice con ella
`analitica/`	Carpeta vacía (solo `.keep`).	Ahí va la app de Streamlit.
Router `/analytics` y `get_current_employee`	Control de rol: 401 sin sesión, 403 si la cuenta no es de vendedor. Un endpoint `/ping` con 9 tests. El comentario del código decía que faltaban «los endpoints de analítica reales».	Los endpoints nuevos van en ese router y heredan el control de acceso sin código extra.
Login único y `EmployeeRoute`	El vendedor que inicia sesión va a `/dashboard`; un cliente no puede entrar a esa ruta.	Se reutiliza tal cual.
`PortalVendedor.jsx`	Placeholder: «El dashboard estará disponible pronto».	Reemplazado por la sección de acceso al dashboard.
`list_best_sellers`	Cálculo de los más vendidos que usa el inicio de la tienda.	Lo reutiliza el endpoint del dashboard.
`seed_order_history`	Unos 6.000 pedidos de 12 meses, con tendencia y estacionalidad.	Datos para probar el dashboard con números reales.

3. Arquitectura: la decisión principal



En naranja, lo nuevo. El dashboard nunca toca la base: todo pasa por el backend.

Decisión: Streamlit consume la API del backend y no se conecta a la base. La alternativa era que la app de Streamlit hiciera las consultas SQL directamente contra Postgres, que es lo más común en los ejemplos de Streamlit. La descarté por estas razones:

- **Un solo control de acceso.** El backend ya sabe distinguir un vendedor de un cliente. Si Streamlit leyera la base, habría que reimplementar el login y los roles en otro lado, y las credenciales de la base quedarían en un servicio más.
- **La lógica de negocio queda en el backend y con tests.** Qué es «stock bajo» o qué ventas cuentan se define en un solo lugar, que se prueba en el CI existente y que comparte con la tienda: los más vendidos del inicio usan la misma función.
- **Streamlit queda como una capa de presentación delgada.** Para desplegarlo en otro lado solo necesita la URL del backend.
- **El proyecto ya iba por ese camino:** el router `/analytics` protegido para vendedores estaba creado y esperando estos endpoints.

El costo es una llamada HTTP más por pantalla, que con 1.089 repuestos no se nota.

4. Qué cambió, por componente

4.1 Backend

Endpoint	Parámetros	Devuelve
`GET /api/v1/analytics/stock`	`low_stock_threshold`: de 0 a 1000, por defecto 5.	Todos los repuestos, de menos a más stock: código, nombre, fabricante, categoría, stock y si tiene stock bajo.
`GET /api/v1/analytics/best-sellers`	`limit`: de 1 a 100, por defecto 10. `days`: opcional; sin valor, cuenta todas las ventas.	Ranking por unidades vendidas, con el stock actual de cada repuesto.

Los dos endpoints responden 401 sin sesión o con el token vencido, y 403 si la cuenta es de un cliente. Archivos: `endpoints/analytics.py`, `schemas/analytics.py` (nuevo), `crud/part.py` (función `list_stock`) y `tests/analytics_test.py`, con 9 tests nuevos.

4.2 App de Streamlit (analitica/)

Archivo	Para qué
`app.py`	La pantalla: login, barra lateral (sesión, umbral de stock bajo, «Actualizar datos»), indicadores y las dos pestañas.
`api.py`	Cliente HTTP del backend. Traduce las respuestas a errores claros: credenciales inválidas, sesión vencida, sin permiso o backend caído.
`datos.py`	Arma las tablas, los estados y los filtros. No usa Streamlit, así que se puede testear solo.

Archivo	Para qué
`tests/`	`datos_test.py` (10 tests) y `app_test.py` (6 tests con `AppTest` de Streamlit y la API simulada).
`.streamlit/config.toml`	Naranja de la tienda en los temas claro y oscuro; sin el botón «Deploy».
`Dockerfile`, `pyproject.toml`, `uv.lock`	Proyecto con uv y Python 3.14, igual que el backend. La imagen no instala las dependencias de desarrollo.

4.3 Frontend

- `PortalVendedor.jsx`: el placeholder pasa a ser la sección «Dashboard de stock y ventas», con el botón «ABRIR DASHBOARD». Abre en una pestaña nueva (con `noopener noreferrer`) la dirección `VITE_DASHBOARD_URL`, que por defecto es `http://localhost:8501`.
- `babel.config.cjs`: en Jest, el reemplazo de `import.meta.env` ahora acepta cualquier variable `VITE_*`, no solo `VITE_API_URL`. Sin esto, Jest no puede cargar el portal.
- `vendedor.test.jsx`: el test que esperaba el texto del placeholder ahora verifica la sección y el enlace.

4.4 Infraestructura y documentación

- `docker-compose.yml`: servicios `dashboard` (puerto 8501, perfil local) y `dashboard-remote` (puerto 8502, perfil remoto), y `VITE_DASHBOARD_URL` en los dos frontends.
- CI: un job `dashboard` que corre los tests de `analitica/` en cada PR, igual que los de backend y frontend.
- `README.md`: una sección nueva, «Dashboard del vendedor», con qué muestra y cómo se corre.

5. Decisiones de diseño y por qué

Decisión	Por qué
Stock bajo = 5 unidades o menos, configurable desde la barra lateral.	Es el mismo corte que usa `seed_order_history` para los repuestos en riesgo. Con el catálogo actual marca 75 con stock bajo y 44 sin stock, una cantidad manejable. Cada vendedor puede cambiarlo sin tocar código.
«Sin stock» separado de «Stock bajo», cada uno con su indicador.	Reponer algo agotado es urgente; uno con stock bajo hay que vigilarlo.
El estado va escrito, además del color de la celda.	Así se distingue también con daltonismo o impreso en blanco y negro. El color solo lo refuerza.
Los más vendidos incluyen los agotados, a diferencia del inicio de la tienda.	Para el vendedor, un más vendido agotado es justo lo primero que hay que reponer. La tienda los oculta porque el cliente no los puede comprar.
Los pedidos cancelados no cuentan.	No son ventas concretadas. Es el criterio que ya tenía la función existente.
Período seleccionable, 12 meses por defecto.	La demanda cambia con la estación del año. 12 meses da la foto anual y 30 días muestra lo reciente.
Barras horizontales de un solo color, con el número al final de cada barra y la tabla debajo.	Las barras son la mejor forma de comparar cantidades ordenadas. Van horizontales porque los nombres son largos, y de un solo color porque hay una sola serie. La tabla da el detalle exacto.
El gráfico muestra nombre y código.	Los nombres no mencionan el auto, así que dos repuestos distintos pueden llamarse igual. Pasa en los datos reales: «Radiador Valeo» COL-0001 y COL-0010 están los dos en el top 10.
Sin caché: se piden los datos en cada interacción, hay un botón «Actualizar datos» y se muestra la hora de los datos.	Así se cumple el criterio de que la información se actualice cuando cambia el stock. Con 1.089 filas la consulta es inmediata, y la hora deja claro a qué momento corresponden los números.

Decisión	Por qué
Login propio en el dashboard, en vez de pasarle el token por la URL.	Un token en la URL queda en el historial y en los logs, y se puede compartir sin querer. El costo es que el vendedor inicia sesión dos veces.
El rol se verifica con `/analytics/ping` al iniciar sesión.	Decide el backend: el dashboard no interpreta el token por su cuenta.
`datos.py` **separado de** `app.py`**.**	La lógica se testea sin levantar Streamlit; `app.py` solo dibuja.
El endpoint de stock no devuelve el precio.	Ninguna de las HU lo pide, y la respuesta queda más chica.
La hora se muestra en la zona de Montevideo.	El contenedor corre en UTC; sin esto, la hora saldría con 3 horas de diferencia.
Tema claro y oscuro, con el naranja de la tienda; los números del gráfico van en gris medio.	Streamlit no adapta el color del texto de los gráficos al tema. Un gris medio se lee sobre los dos fondos (comprobado con capturas).
Sin el botón «Deploy» de Streamlit.	Es una herramienta interna para vendedores, no para desarrolladores.
Un job de CI propio, con la API simulada en los tests.	Los tests del dashboard corren en cada PR sin levantar el backend ni la base.

6. Criterios de aceptación

HU 5.1: Acceder al dashboard

Criterio	Cómo se cumple	Verificado con
Debe existir una sección de acceso.	Botón «Abrir dashboard» en `/dashboard`, ruta que solo ven los vendedores. El dashboard corre en el puerto 8501.	Test de Jest
Solo acceden vendedores o administradores.	Los endpoints exigen cuenta de vendedor, y el dashboard la verifica al iniciar sesión.	Tests 401/403 del backend; test «un vendedor entra»
Un cliente no debe acceder.	La API le responde 403; el dashboard muestra «Esta cuenta no tiene acceso al dashboard» y no guarda la sesión.	`app_test.py` y captura con la cuenta de Ana
Los datos vienen del sistema.	Todo sale de la base a través de la API; nada está escrito a mano.	Prueba con los seeds

Sobre «administradores»: el sistema hoy tiene solo dos tipos de cuenta, cliente y vendedor (`employee`). Si se agrega un rol de administrador, alcanza con aceptarlo en `get_current_employee`.

HU 5.2: Visualizar stock actual

Criterio	Cómo se cumple	Verificado con
Se muestra el stock actual.	Pestaña «Stock actual» con los 1.089 repuestos, de menos a más stock.	Test del endpoint y captura
Como mínimo, nombre y cantidad.	Columnas Repuesto y Stock, más Código, Fabricante, Categoría y Estado.	Captura
Se identifica el stock bajo.	Estado «Stock bajo» o «Sin stock», escrito y con color; indicadores arriba; filtro por estado.	Tests de `datos.py` y del endpoint
Se actualiza con los cambios de stock.	Sin caché: cada interacción vuelve a consultar.	`test_stock_refleja_los_cambios_de_stock`
Se pueden consultar los distintos repuestos.	Búsqueda por nombre, código o fabricante, y filtro por categoría.	Tests de los filtros

HU 5.3: Visualizar productos más vendidos

Criterio	Cómo se cumple	Verificado con
Existe una sección de más vendidos.	Pestaña «Más vendidos».	Test de la app y captura
Ordenados por unidades vendidas.	Suma de unidades de mayor a menor; los empates se ordenan por id, así el orden no cambia entre consultas.	`test_mas_vendidos_ordena_por_unidades_vendidas`
Se muestran las unidades vendidas.	Número al final de cada barra y columna «Unidades vendidas».	Captura
Sale de las ventas registradas.	Ítems de pedidos reales, sin contar los cancelados.	Test de cancelados y de período
Se identifica fácilmente la mayor demanda.	Gráfico ordenado, con top N y período elegibles.	Captura

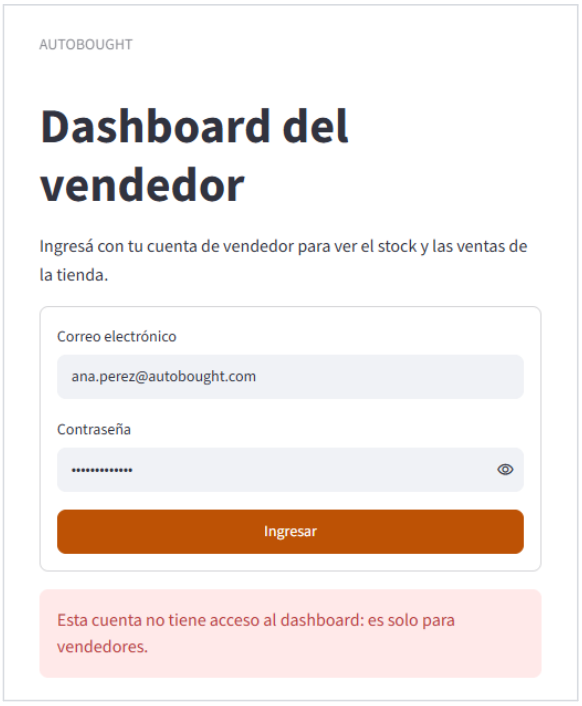
7. Pruebas

Suite	Resultado
Backend (pytest)	384 tests pasan; 98 % de cobertura total y 100 % en `analytics.py`. 9 tests nuevos.
Dashboard (pytest)	16 tests pasan: 10 de `datos.py` y 6 de la app (login, cliente rechazado, credenciales incorrectas, sesión vencida, cerrar sesión).
Frontend	340 tests de Jest pasan; oxlint sin errores; el build funciona.
Prueba manual	Backend real sobre una base SQLite con los tres seeds (1.089 repuestos y 6.011 pedidos). Capturas con Playwright en tema claro, oscuro y ancho de celular.

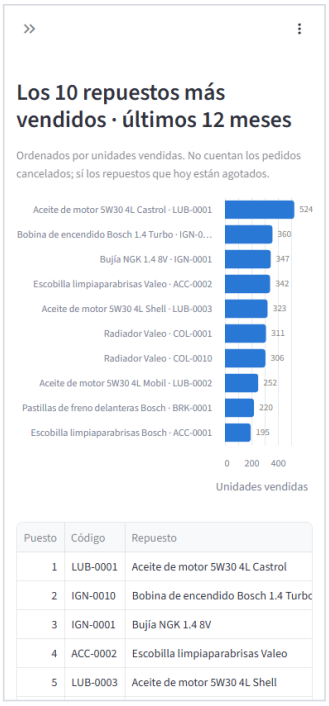
La revisión visual encontró cinco problemas que los tests no detectan, y se corrigieron:

- El tema oscuro no se aplicaba: un `[theme]` suelto en el config deja la app siempre en claro. Se separó en `[theme.light]` y `[theme.dark]`.
- En tema oscuro, los números de las barras eran negros sobre fondo oscuro y no se veían.
- El correo del vendedor salía como un enlace `mailto`.
- El eje del gráfico tenía una marca cada 20 unidades y no se leía.
- Aparecía el botón «Deploy» de Streamlit.

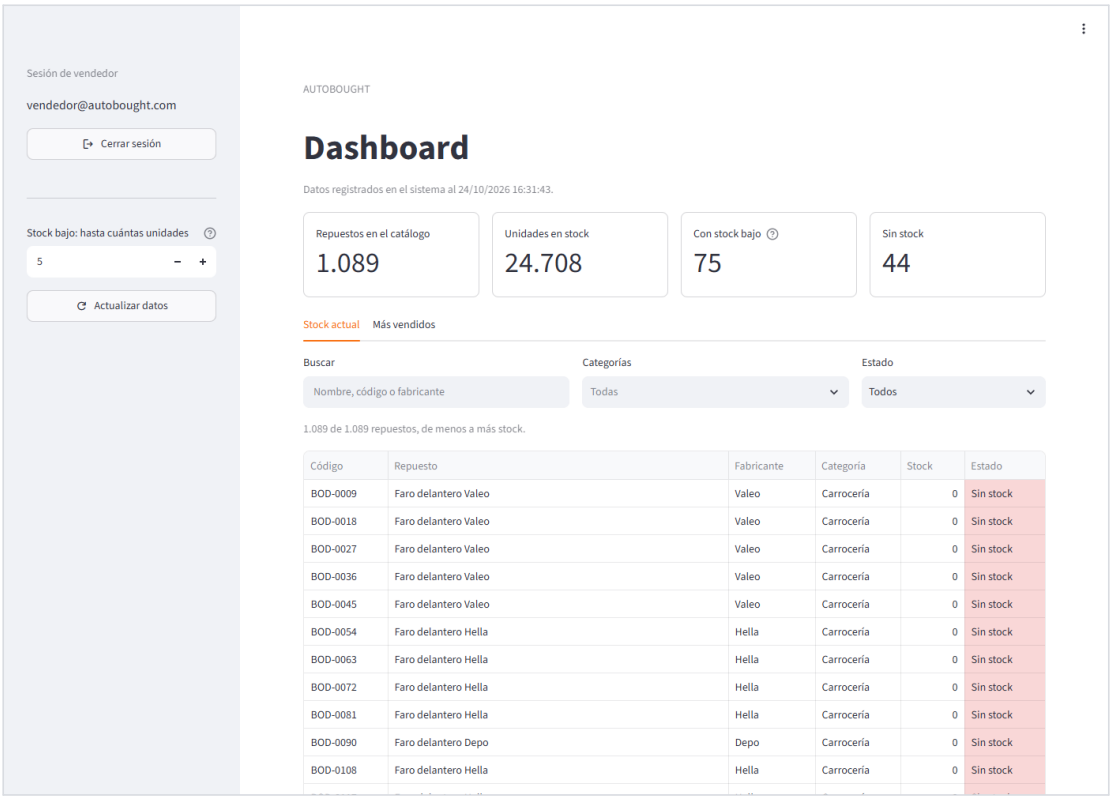
8. Capturas



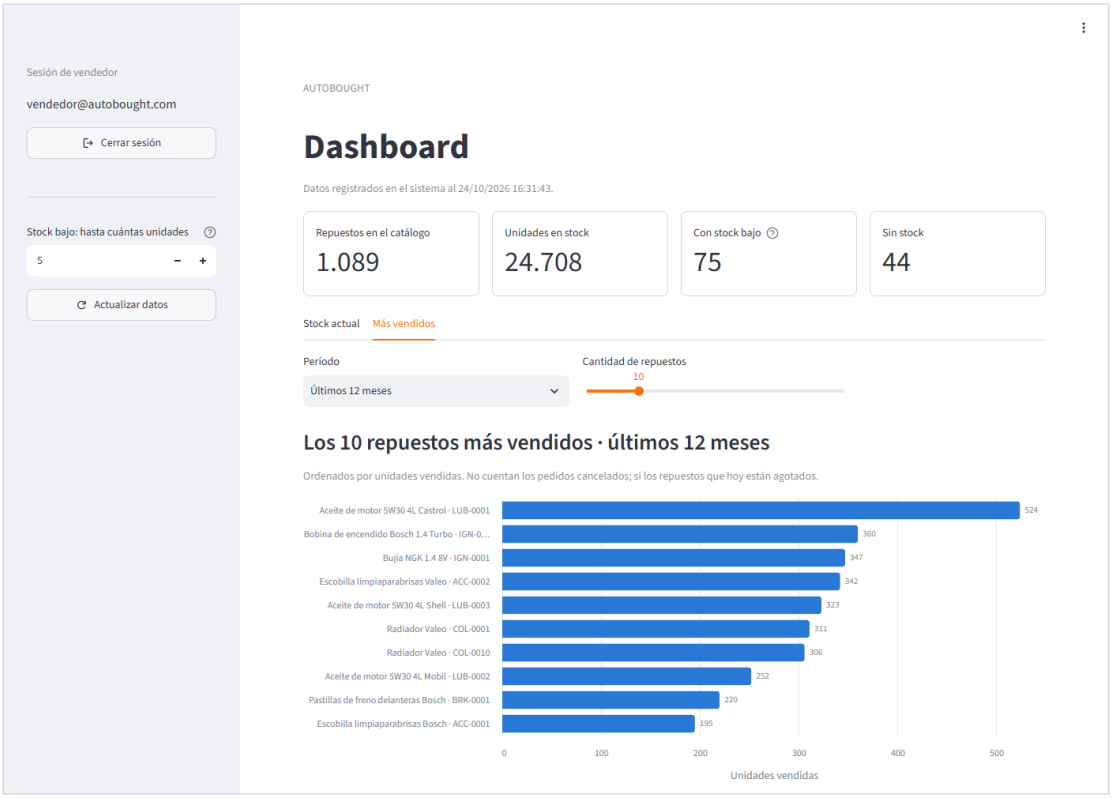
Un cliente (Ana) intenta entrar: el backend le responde 403 y el dashboard no lo deja pasar del login.



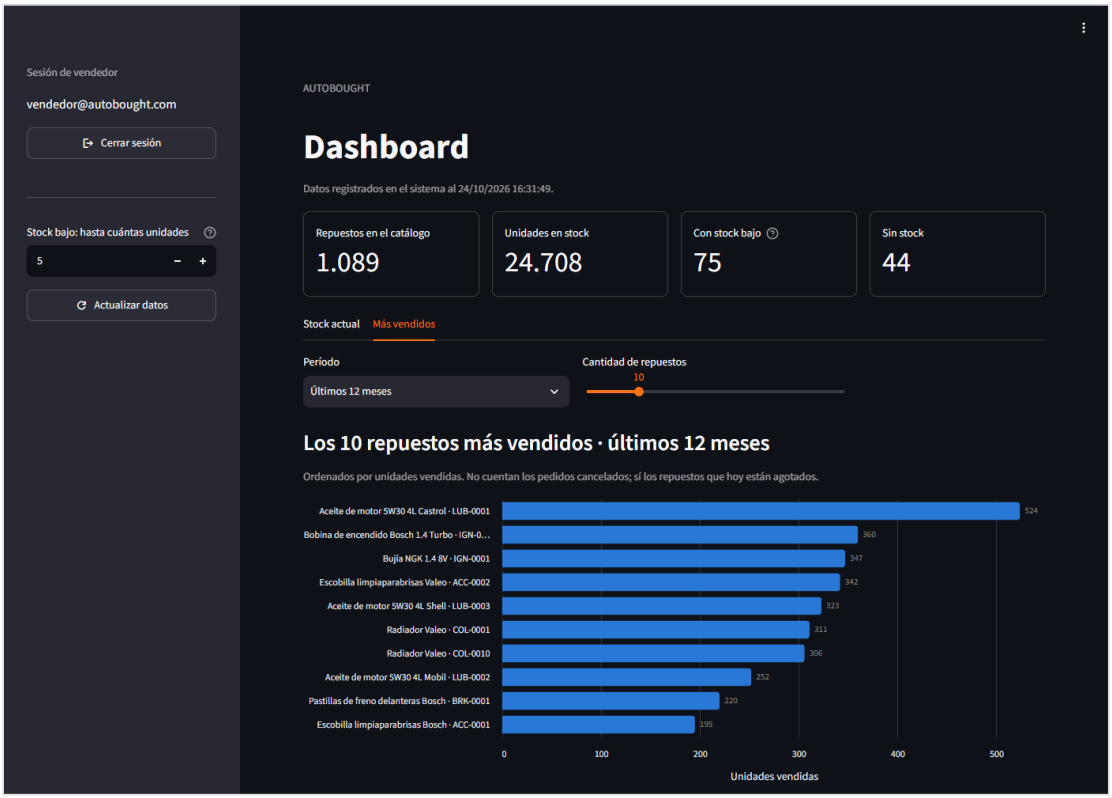
En el celular los indicadores se apilan y el gráfico se adapta; los nombres largos le quitan ancho a las barras.



Stock actual: indicadores arriba, filtros en una fila y la tabla de menos a más stock, con el estado escrito.



Más vendidos de los últimos 12 meses: barras ordenadas con las unidades al final, nombre y código.



La misma vista en tema oscuro.

9. Cómo probarlo

Con Docker, en la raíz del repo, con el `.env` completo y los tres seeds cargados (ver README):

```
docker compose up -d --build
# Tienda: http://localhost:5173 (vendedor@autobought.com / Ventas2026x)
# Dashboard: http://localhost:8501
```

Sin Docker, con el backend corriendo en el puerto 8000:

```
cd analitica
uv sync
uv run streamlit run app.py
uv sync --group dev && uv run pytest # tests
```

10. Pendientes y limitaciones

- **Docker:** no se pudo construir la imagen porque Docker no estaba corriendo. La configuración se validó con `docker compose config`; antes del PR conviene correr `docker compose up -d --build`.
- **Producción:** Vercel no puede correr Streamlit. Hay que desplegarlo aparte (Streamlit Community Cloud u otro servicio), apuntar `AUTOBOUGHT_API_URL` al backend de producción y configurar `VITE_DASHBOARD_URL` en `frontend/.env.production`. No hace falta tocar CORS, porque las llamadas salen del servidor de Streamlit y no del navegador.
- **Doble inicio de sesión:** el vendedor vuelve a iniciar sesión en el dashboard. Para evitarlo con seguridad haría falta un mecanismo de traspaso de sesión (por ejemplo, un código de un solo uso), que queda fuera de estas HU.
- **Rol administrador:** no existe en el modelo de datos; ver la sección 6.
- **Celular:** el dashboard funciona, pero está pensado sobre todo para la computadora.